CS59300CVD Assignment 1

Due date: Friday, Feb. 6, 2026 11:59 PM

This assignment was adapted from Svetlana Lazebnik's course on computer vision.

Homework Policy: Each student should write up their solutions independently, no copying of any form is allowed. If you have used online resources to help with answering the questions, but have come up with your own answers, please properly reference the resources. You are allowed to use online resources, including any AI, as long as the answer is your own and properly acknowledged. Please refer to the course website for the policy on academic honesty. Finally, you **must use the provided LaTeX template** to typeset the report.

Additional Computing Resource: This assignment does not require heavy computation and can be completed on a personal laptop. However, we also provide access to Scholar, a small computer cluster equipped with a job scheduler for managing batch jobs. You may log in to Scholar and submit a job following the provided instructions. Please note that **Scholar may only be used for course-related work**.

Assignment Details

Objective: This programming section is designed to get you familiar with PyTorch and image manipulation basics, including `torch.tensor`, basic tensor manipulation functions under `torch`, and `PIL` and `numpy`. We assume basic knowledge of setting up a conda environment and installing packages. In your code, please document the environment and package version for reproducibility.

Background: Sergei Mikhailovich Prokudin-Gorskii (1863–1944) was a photographer who recorded three exposures of every scene onto a glass plate using a red, green, and blue filter. Back then, there was no way to print such photos, and they had to be displayed using a special projector. In this assignment, we will take the digitized Prokudin-Gorskii glass plate images and automatically produce a color image with as few visual artifacts as possible. The inputs and outputs of the process are shown in the figure below (Note, the image quality will be lower in the assignment to save compute.):

To do this, you will need to extract the three color channel images, place them on top of each other, and align them so that they form a single RGB color image automatically. We have provided a starter code for you!

Method description

Please follow the steps below for the implementation.

1. **Step 1:** Your program should divide the image into three parts (channels) by removing the boundary and padding the images to the same size. Be creative about this!

2. **Step 2:** Align two channels to the third (you should try different orders of aligning the channels and figure out which one works the best).

The easiest way to align the parts is to exhaustively search over a window of possible displacements (say $[-15, 15]$ pixels independently for the x and y axis), score each one using some image matching metric, and take the displacement with the best score.

One could use several possible metrics for matching; we will consider the following:

1. ℓ_2-norm of the pixel differences of the two channels, also known as the sum of squared differences (SSD). In our case, the images to be matched do not have the same brightness values (they are different color channels), so a clever metric might work better.

2. Normalized cross-correlation (NCC), which is the dot product between the two images normalized to have zero mean and unit norm.

3. Structural similarity (SSIM), the implementation is provided to you. Please refer to "Image quality assessment: from error visibility to structural similarity" by Wang et al. for details.

Test your alignment algorithm on the provided images. Note that the filter order for all files from top to bottom is **BGR**.

What to include in the report? (10 points)

1. Mathematically formulate the alignment procedure as an optimization problem. Clearly define all introduced notations. (2 points)

2. A brief description of your implementation, focusing especially on the more "non-trivial" or interesting parts of the solution. Explain your thought process. What implementation choices did you make, and how did they affect the quality of the result and the speed of computation? What are some artifacts and/or limitations of your implementation, and what are possible reasons for them? (5 points)

3. For each input image, you must include in your report the colorized output and the (x, y) displacement vectors used to align the channels. (3 points)

4. While code is not explicitly graded, poorly commented or written code will result in points deducted from the previous three parts (e.g., if the grader cannot understand what was implemented).

What to include in the code?

The code should contain a `README.md` file that describes how to run and reproduce your result. This includes the packages installed and their versions. We expect adequately commented code. To help with readability, please follow a style guide and use a linter (e.g., PEP8 is usually a good choice).

How to submit?

- You should have received an e-mail with the link to access Gradescope. If you haven't, let the TAs know.
- For your pdf file, use the naming convention `username_hw#.pdf`. For example, a TA with username `alice` would name her pdf file for HW1 as `alice_hw1.pdf`.
- Use the provided LaTeX template to typeset the assignment by editing the tex files under `student_response/`.
- After uploading your submission to Gradescope, mark each page to identify which question is answered on the page (Gradescope will facilitate this).
- For your code, use the naming convention `username_hw#.zip`. The code can be submitted via Gradescope under **assignment1 programming**.